ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

PROJET DE FIN MATIERE

Filière :

Ingénierie Informatique et Réseaux : IASDT GRP 1

Intitulé du Projet

SentimentAI Morocco

Pipeline Big Data & Analyse de Sentiment en Temps Réel

Encadré par : -M. BADIR Hassan

Réaliser Par :

Harboul Zaid
Chelle Zakariae
Moh Ousellam Mohamed
Isakrouhen Yasmine

Membres du jury:

Mr. BADIR HASSAN

Année Universitairre : 2026-2027

2ème Année-Cycle d'ingénieure

Remerciements

Nous souhaitons exprimer notre sincère gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce projet. Leur soutien, leurs conseils et leur bienveillance ont été des moteurs essentiels tout au long de cette aventure académique et technique.

Nous tenons à remercier tout particulièrement M. BADIR Hassan, notre encadrant, pour sa disponibilité constante, ses orientations précieuses et ses retours constructifs. Sa maîtrise des technologies Big Data et son exigence académique nous ont permis de repousser nos limites et de produire un travail dont nous sommes fiers. Ses encouragements et sa pédagogie exemplaire ont été une source d'inspiration permanente.

Nous adressons également notre vive reconnaissance aux membres du jury, dont l'expertise et l'esprit critique ont enrichi notre réflexion et renforcé la rigueur de notre démarche. Nous sommes honorés d'avoir eu l'opportunité de présenter notre travail devant un comité aussi qualifié.

Notre gratitude s'étend à l'ensemble de l'équipe pédagogique de l'EMSI Tanger, pour l'environnement d'apprentissage dynamique et stimulant qu'elle met à notre disposition. Les ressources académiques, les laboratoires et les enseignements dispensés nous ont fourni le socle de compétences nécessaire à la réalisation de ce projet ambitieux.

Enfin, nous exprimons notre profonde reconnaissance à nos familles et à nos proches, pour leur soutien indéfectible, leur patience et leurs encouragements tout au long de cette période de travail intense. Ce projet leur doit autant qu'à nous-mêmes.

Dédicace

À nos parents bien-aimés,

Pour votre soutien indéfectible, votre patience infinie, vos sacrifices constants et votre amour inconditionnel. Vous méritez toutes les louanges et reconnaissances du monde. Grâce à vous, nous avons eu la chance de poursuivre nos études dans les meilleures conditions et de devenir les ingénieurs que nous aspirons à être. Que Dieu vous protège, vous bénisse et vous récompense.

À nos frères, sœurs et amis,

Pour leur affection sincère, leur présence bienveillante et leurs encouragements continus. Votre soutien moral a été une force précieuse qui nous a aidés à surmonter les moments de doute et à persévérer jusqu'à l'aboutissement de ce travail.

À nos camarades de promotion,

Pour le partage de connaissances, les échanges enrichissants et la solidarité dont vous avez fait preuve tout au long de notre parcours commun à l'EMSI Tanger. Ces années d'études partagées resteront gravées dans nos mémoires.

Nous dédions ce travail à tous ceux qui croient en la valeur de la connaissance,
et qui œuvrent chaque jour pour un avenir meilleur grâce à la technologie.

Résumé

Ce projet de fin de matière, intitulé SentimentAI Morocco, porte sur la conception et le développement d'un système intelligent de suivi en temps réel de l'opinion publique et des tendances numériques au Maroc. La solution repose sur une architecture Big Data complète, intégrant un pipeline de collecte automatisé, un moteur d'analyse de sentiment multiculturel et un tableau de bord interactif haute performance.

La collecte de données est assurée par un parseur RSS développé en Python, exploitant le flux XML de r/Morocco/new.rss via xml.etree.ElementTree afin de contourner les limitations de l'API JSON officielle de Reddit (blocages HTTP 403). Ces données brutes sont ensuite traitées par un pipeline MapReduce distribué sur un cluster Apache Hadoop déployé sous Docker Compose, articulé en cinq jobs séquentiels exécutés via l'API Hadoop Streaming : nettoyage et normalisation des données, scoring NLP par un modèle de machine learning scikit-learn (TF-IDF + Régression Logistique), détection des hashtags dominants par pondération TF-IDF, détection d'anomalies temporelles, et calcul de la vélocité de tendance géo-temporelle — contribution algorithmique originale du projet.

Un tableau de bord interactif a été développé en HTML/CSS/JavaScript vanilla, animé par GSAP et alimenté par une API FastAPI. Il intègre une carte choroplèthe du Maroc (Leaflet.js), un fil de publications en temps réel, un graphique de vélocité de sentiment (Chart.js), un visualiseur de pipeline MapReduce animé et un panneau d'alertes avec classement dynamique. L'ensemble du système fonctionne avec des données réelles ou simulées, et est facilement commutable entre les deux modes.

Ce projet illustre l'importance des technologies Big Data et de l'intelligence artificielle dans la transformation de flux de données non structurées en outils stratégiques d'aide à la décision et de veille numérique, au service de la transformation digitale du Maroc.

Mots-clés : Big Data, Apache Hadoop, Hadoop Streaming, Analyse de sentiment, NLP, scikit-learn, FastAPI, GSAP, Leaflet.js, Python, Temps réel, Maroc.

Abstract

This end-of-course project, titled SentimentAI Morocco, focuses on the design and development of an intelligent system for real-time monitoring of public opinion and digital trends in Morocco. The solution is built on a comprehensive Big Data architecture, integrating an automated data collection pipeline, a multicultural sentiment analysis engine, and a high-performance interactive dashboard.

Data collection is handled by a Python RSS feed parser leveraging the r/Morocco/new.rss XML endpoint via xml.etree.ElementTree, bypassing the HTTP 403 rate-limiting blocks imposed by Reddit's official JSON API. These raw data are then processed by a distributed MapReduce pipeline running on an Apache Hadoop cluster deployed via Docker Compose, structured around five sequential jobs executed through the Hadoop Streaming API: data cleaning and normalisation, NLP sentiment scoring using a pre-trained scikit-learn model (TF-IDF vectoriser and Logistic Regression classifier), dominant hashtag detection via TF-IDF weighting, temporal anomaly detection, and geo-temporal trend velocity computation — an original algorithmic contribution of the project.

An interactive dashboard was developed in vanilla HTML/CSS/JavaScript, animated by GSAP and powered by a FastAPI backend. It features a Morocco choropleth map (Leaflet.js), a real-time post feed, a sentiment velocity chart (Chart.js), an animated MapReduce pipeline visualiser, and a dynamic alert panel with live ranking.

This project demonstrates the importance of Big Data technologies and artificial intelligence in transforming unstructured data streams into strategic decision-support and digital intelligence tools, in service of Morocco's digital transformation.

Keywords: Big Data, Apache Hadoop, Hadoop Streaming, Sentiment Analysis, NLP, scikit-learn, FastAPI, GSAP, Leaflet.js, Python, Real-time, Morocco.

Sommaire

Chapitre 1 : Contexte Général du Projet et État de l'Art ..... *Error! Bookmark not defined.*

Introduction ..... **Error! Bookmark not defined.**

1. Big Data et Analyse de Sentiment : Fondements Théoriques 9

1.1 Le paradigme Big Data 9

1.2 L'analyse de sentiment et le Traitement du Langage Naturel 10

2. Contexte du Projet 10

3. Problématique 11

4. Objectifs du Projet 11

4.1 Objectifs fonctionnels 11

4.2 Objectifs techniques 11

Conclusion 12

Chapitre 2 : Analyse des Besoins et Conception de l'Architecture ..... *Error! Bookmark not defined.*

Introduction 13

1. Analyse des Besoins 13

1.1 Besoins fonctionnels 13

1.2 Besoins non fonctionnels 14

2. Architecture Générale de la Solution 14

2.1 Vue d'ensemble en couches 14

2.2 Flux de données 15

2.3 Architecture du pipeline MapReduce 15

3. Conception Détaillée des Composants Clés 16

3.1 Le module d'inférence géographique 16

3.2 Le Job 5 — Vélocité de tendance géo-temporelle (contribution originale) 16

4. Outils et Technologies Utilisés 16

Conclusion 17

Chapitre 3 : Implémentation et Développement de la Solution *Error! Bookmark not defined.*

Introduction 18

1. Pipeline MapReduce — Architecture Hadoop Distribué 18

1.1 Structure du projet Python 18

1.2 Job 1 — Nettoyage et normalisation 18

1.3 Job 2 — Scoring NLP par lexique 19

1.4 Job 3 — Détection des hashtags dominants par TF-IDF 19

1.5 Job 4 — Détection d'anomalies temporelles 20

1.6 Job 5 ★ — Vélocité de tendance géo-temporelle 20

2. Backend Python — FastAPI 20
2.1 Structure et organisation 20
2.2 Scraper et gestion des APIs sociales 20
2.3 Endpoints REST 21
3. Tableau de Bord — Frontend JavaScript 21
3.1 Architecture événementielle 21
3.2 Pipeline Visualiseur (Panel 1) 21
3.3 Carte choroplèthe Maroc (Panel 2) 22
3.4 Animations GSAP et performance 22
3.5 Le classement dynamique (Flip pattern) 22
Conclusion 22
Chapitre 4 : Résultats, Analyse et Évaluation de la Solution ... *Error! Bookmark not defined.*
Introduction 24
1. Résultats du Pipeline MapReduce 24
1.1 Données de test 24
1.2 Exemple de sortie — Job 5 Vélocité 25
2. Évaluation du Module NLP 25
2.1 Méthodologie d'évaluation 25
2.2 Analyse des résultats NLP 26
3. Présentation du Tableau de Bord 26
3.1 Vue d'ensemble 26
3.2 Démonstration interactive — séquence recommandée 26
4. Évaluation Critique de la Solution 30
4.1 Points forts 30
4.2 Limitations et perspectives 30
Conclusion 31
Annexe A — Structure du projet 34
Annexe B — Commandes de démarrage 35
Annexe C — Format des données JSON 35
Annexe D — Bibliographie 35

Listes des figures

Figure 1 : Architecture générale du système SentimentAI Morocco .....
Figure 2 : Pipeline MapReduce — Vue d'ensemble des 5 jobs .....
Figure 3 : Flux de données : de la collecte à la visualisation .....
Figure 4 : Modèle de données — Format JSON des publications sociales .....
Figure 5 : Diagramme de l'inférence géographique par mots-clés .....
Figure 6 : Formule de vélocité de tendance géo-temporelle .....
Figure 7 : Architecture FastAPI — Endpoints et flux de réponse .....
Figure 8 : Vue du tableau de bord — Panneau pipeline visualiseur .....
Figure 9 : Vue du tableau de bord — Carte choroplèthe du Maroc .....
Figure 10 : Vue du tableau de bord — Fil de publications en temps réel .....
Figure 11 : Vue du tableau de bord — Graphique de vélocité .....
Figure 12 : Vue du tableau de bord — Panneau alertes et classement .....
Figure 13 : Résultats d'évaluation — Scores F1 par région .....
Figure 14 : Comparaison simulation vs données réelles

Introduction générale

À l'ère du numérique, les réseaux sociaux génèrent chaque seconde des millions de publications reflétant les opinions, les émotions et les préoccupations d'une société. Cette explosion de données non structurées représente un défi technologique majeur, mais également une opportunité sans précédent pour les acteurs économiques, politiques et académiques qui souhaitent comprendre et anticiper les dynamiques de l'opinion publique.

C'est dans ce contexte que s'inscrit SentimentAI Morocco, un projet ambitieux visant à concevoir et développer une plateforme Big Data intelligente dédiée au suivi en temps réel du climat numérique au Maroc. En exploitant les données massives issues de Reddit, le système analyse automatiquement le sentiment exprimé par les internautes marocains, détecte les tendances émergentes et alerte sur les anomalies de manière proactive.

La pertinence de ce projet est renforcée par le contexte marocain spécifique : un pays à la croisée des influences culturelles arabes, berbères et françaises, où les interactions numériques mêlent dialecte darija, arabe classique et français. Cette réalité linguistique impose des défis particuliers en matière de traitement du langage naturel, que notre solution aborde par l'intégration d'un modèle scikit-learn pré-entraîné (TfidfVectorizer + Régression Logistique) directement dans le Mapper Hadoop, en conservant le lexique AFINN-111 enrichi de termes marocains comme mécanisme de repli déterministe.

Sur le plan technique, le projet mobilise un ensemble cohérent de technologies Big Data modernes : Apache Hadoop via Docker Compose pour le traitement véritablement distribué à grande échelle, l'API Hadoop Streaming pour l'exécution des scripts Python mapper.py et reducer.py au sein de l'écosystème HDFS, FastAPI pour l'orchestration et l'exposition des données, et un tableau de bord HTML/JavaScript animé par GSAP pour la visualisation temps réel. L'originalité algorithmique du projet réside dans le Job 5 du pipeline MapReduce, qui introduit le concept de vélocité de tendance géo-temporelle : plutôt que de simplement mesurer le sentiment actuel, le système calcule sa vitesse de changement par région et par semaine, permettant ainsi la détection précoce des crises avant qu'elles n'atteignent leur pic.

Ce rapport est structuré en quatre chapitres complémentaires. Le premier chapitre présente le contexte général du projet, les technologies mobilisées et la problématique adressée. Le deuxième chapitre détaille l'analyse des besoins fonctionnels et non fonctionnels ainsi que l'architecture décisionnelle conçue. Le troisième chapitre décrit l'implémentation technique de chaque composant du système. Le quatrième chapitre présente et analyse les résultats obtenus, avant de conclure sur les apports et les perspectives du projet.

1

Chapitre 1 : Contexte Général du Projet et État de l'Art

Introduction :

Ce premier chapitre pose les fondations conceptuelles et contextuelles du projet SentimentAI Morocco. Il présente les paradigmes technologiques sur lesquels repose la solution, analyse l'environnement dans lequel elle s'inscrit, identifie la problématique adressée et définit les objectifs visés.

1. Big Data et Analyse de Sentiment : Fondements Théoriques

1.1 Le paradigme Big Data

Le Big Data désigne l'ensemble des technologies et méthodologies permettant de collecter, stocker, traiter et analyser des volumes de données massifs, dépassant les capacités des systèmes de traitement traditionnels. Il se caractérise par les cinq dimensions du modèle des 5V :

- **Volume :** des téraoctets, voire des pétaoctets de données générées en continu.
- **Vélocité :** des flux de données produits et consommés en temps quasi-réel.
- **Variété :** des formats hétérogènes (texte, image, audio, JSON, CSV...).
- **Véracité :** la nécessité de filtrer les données bruit et incomplètes.
- **Valeur :** la transformation des données brutes en insights stratégiques actionnables.

Le paradigme MapReduce, introduit par Google (Dean & Ghemawat, 2008), décompose tout traitement en deux phases : une phase Map qui filtre et transforme les données en paires clé-valeur, et une phase Reduce qui agrège les résultats. Dans notre implémentation, la phase Map est distribuée sur un cluster Apache Hadoop (1 NameNode, 1 ResourceManager, 1 HistoryServer, 4 DataNodes) via l'API Hadoop Streaming, qui exécute directement les scripts

Python mapper.py et reducer.py au sein de l'écosystème HDFS. Le NameNode orchestre la distribution des fichiers .jsonl bruts vers les DataNodes, où les Mappers s'exécutent en parallèle et écrivent les paires clé-valeur (Région → Score de sentiment) en HDFS. Cette architecture garantit la tolérance aux pannes, la réplication des données et une scalabilité horizontale réelle.

1.2 L'analyse de sentiment et le Traitement du Langage Naturel

L'analyse de sentiment (ou opinion mining) est un domaine de la linguistique computationnelle visant à identifier automatiquement l'opinion exprimée dans un texte : positive, négative ou neutre. Elle mobilise des techniques de Traitement du Langage Naturel (NLP) parmi lesquelles :

- **Approches lexicales :** utilisation de dictionnaires de sentiment (AFINN, SentiWordNet, VADER) qui associent un score numérique à chaque terme.
- **Approches par apprentissage machine :** classification supervisée (Naive Bayes, SVM, BERT) entraînée sur des corpus annotés.
- **Approches hybrides :** combinaison des deux pour améliorer la robustesse sur des textes dialectaux ou multilingues.

Le contexte marocain présente des défis linguistiques spécifiques : la darija (dialecte arabe marocain), l'arabe classique et le français coexistent dans les publications sociales, souvent au sein d'un même message (code-switching). Notre solution adopte une approche hybride à deux niveaux : un modèle de machine learning pré-entraîné (TfidfVectorizer + Régression Logistique, fichier Model final.joblib) constitue le moteur d'inférence principal, tandis que le dictionnaire AFINN-111 enrichi de termes marocains intervient exclusivement comme mécanisme de repli déterministe en cas de défaillance du modèle ML.

2. Contexte du Projet

SentimentAI Morocco s'inscrit dans le cadre du module Big Data et Traitement Distribué de la filière Ingénierie Informatique et Réseaux (IASDT) de l'EMSI Tanger. Le projet répond à une problématique réelle et d'actualité : la nécessité pour les acteurs marocains (entreprises, institutions, médias) de disposer d'outils de veille numérique capables d'analyser en temps réel le sentiment exprimé sur les réseaux sociaux par rapport aux événements nationaux.

La montée en puissance des réseaux sociaux au Maroc est significative : avec plus de 22 millions d'utilisateurs d'internet et un taux de pénétration des réseaux sociaux dépassant 50% de la population, Reddit constituent des baromètres précieux du climat d'opinion, notamment lors d'événements sportifs (CAN 2025, Coupe du Monde 2030), politiques ou économiques majeurs.

3. Problématique

Comment exploiter des données massives et hétérogènes issues des réseaux sociaux pour identifier en temps réel les corrélations entre régions géographiques, périodes temporelles et évolution du sentiment public marocain, et produire un système d'alerte précoce sur les tendances émergentes ?

Cette problématique soulève plusieurs défis techniques interdépendants :

- Comment collecter en continu des données massives depuis plusieurs API sociales sans interruption de service ?
- Comment analyser le sentiment dans un contexte multilingue et dialectal (darija, arabe, français) avec une précision satisfaisante ?
- Comment détecter les tendances non pas par leur valeur absolue, mais par leur vitesse de changement, pour anticiper les crises ?
- Comment visualiser ces informations de manière fluide, intuitive et temps réel, adaptée aux besoins d'analyse stratégique ?

4. Objectifs du Projet

4.1 Objectifs fonctionnels

- Collecter automatiquement des publications sociales depuis Reddit ciblant les 8 régions du Maroc.
- Analyser le sentiment de chaque publication (score entre -1 et +1) en tenant compte du multilinguisme marocain.
- Détecter les hashtags dominants par région et par période en éliminant les tags globalement viraux.
- Identifier les anomalies temporelles : régions dont l'activité dépasse de 50% la moyenne des 6 semaines précédentes.
- Calculer la vélocité de tendance géo-temporelle et produire un classement des régions par urgence d'alerte.
- Visualiser l'ensemble de ces informations sur un tableau de bord interactif et animé en temps réel.

4.2 Objectifs techniques

- Déployer un cluster Apache Hadoop distribué (1 NameNode + 4 DataNodes) via Docker Compose et exécuter un pipeline MapReduce en 5 jobs séquentiels via l'API Hadoop Streaming (mapper.py et reducer.py).
- Développer un backend Python FastAPI servant de couche API entre le pipeline et le tableau de bord.
- Créer un tableau de bord HTML/CSS/JavaScript vanilla avec des animations GSAP de qualité professionnelle.

- Garantir une architecture modulaire permettant le passage transparent entre données simulées et données réelles.
- Documenter l'ensemble du système (docstrings Python, JSDoc, README) pour faciliter la maintenance.

Conclusion

Ce chapitre a posé les bases théoriques et contextuelles du projet SentimentAI Morocco. Nous avons établi la pertinence de combiner Big Data, NLP et visualisation temps réel pour répondre à un besoin concret de veille numérique dans le contexte marocain. Le chapitre suivant détaille l'analyse des besoins et l'architecture technique conçue pour répondre à cette problématique.

2

Chapitre 2 : Analyse des Besoins et Conception de l'Architecture

Introduction

Ce chapitre présente l'analyse des besoins fonctionnels et non fonctionnels du système SentimentAI Morocco, ainsi que les choix d'architecture et de conception qui ont guidé son développement. Une attention particulière est portée à la modularité, à la scalabilité et à la capacité du système à fonctionner en mode temps réel.

1. Analyse des Besoins

1.1 Besoins fonctionnels

L'analyse des besoins fonctionnels a été conduite selon la méthode SMART (Spécifique, Mesurable, Atteignable, Réaliste, Temporel). Elle a identifié les fonctionnalités suivantes, classées par priorité :

Priorité	Fonctionnalité	Description
Haute	Collecte automatique	Scraper Reddit toutes les 60 secondes
Haute	Scoring NLP	Attribution d'un score de sentiment [-1, +1] à chaque post
Haute	Pipeline MapReduce	5 jobs Python enchaînés via driver.py
Haute	Carte choroplèthe	Heatmap Maroc mise à jour en temps réel

Priorité	Fonctionnalité	Description
Moyenne	Détection anomalies	Alerte si activité > 1,5× moyenne 6 semaines
Moyenne	Vélocité de tendance	Calcul Δsentiment/Δt par région et semaine
Moyenne	Hashtag TF-IDF	Top hashtags locaux après pondération globale
Faible	Mode simulation	Données simulées cohérentes pour les démonstrations

1.2 Besoins non fonctionnels

- **Performance :** le tableau de bord doit se rafraîchir en moins de 5 secondes. Le pipeline MapReduce doit traiter 10 000 posts en moins de 2 minutes.
- **Disponibilité :** le backend FastAPI doit rester opérationnel même en cas d'indisponibilité des APIs Reddit.
- **Scalabilité :** l'architecture Hadoop distribuée doit supporter une montée en charge horizontale par ajout de DataNodes supplémentaires au cluster.
- **Maintenabilité :** chaque composant est documenté (docstrings Python, JSDoc) et suit les principes SOLID.
- **Sécurité :** les credentials API sont stockés dans config.yaml, jamais en dur dans le code source.
- **Portabilité :** le tableau de bord fonctionne sans serveur local (double-clic sur index.html) et sans framework.

2. Architecture Générale de la Solution

2.1 Vue d'ensemble en couches

Le système SentimentAI Morocco est organisé en trois couches fonctionnelles indépendantes et faiblement couplées, communicant via des formats de données standardisés (JSON, JSONL, CSV) :

Couche	Composants	Rôle
Couche Collecte	scraper.py, region_inferrer.py	Ingestion des données sociales brutes depuis les APIs
Couche Traitement	Cluster Hadoop (NameNode + 4 DataNodes), Hadoop Streaming (mapper.py, reducer.py), sentiment_processor.py	Nettoyage, scoring NLP, agrégation et analyse

Couche	Composants	Rôle
Couche Présentation	FastAPI main.py, Dashboard HTML/JS/CSS	Exposition des résultats et visualisation temps réel

2.2 Flux de données

Le flux de données traverse le système de manière unidirectionnelle, du scraper jusqu'au tableau de bord, selon la séquence suivante :

- Le scraper.py collecte les publications toutes les 60 secondes via le parseur RSS (r/Morocco/new.rss) et les écrit en /data/raw/YYYY-MM-DD.jsonl.
- Le sentiment_processor.py est déclenché immédiatement après chaque cycle de scraping. Il score les publications non encore traitées et produit les fichiers JSON dans /data/processed/.
- FastAPI lit ces fichiers JSON et les expose via 6 endpoints REST avec en-tête Cache-Control: no-store.
- Le DataPoller JavaScript interroge les 6 endpoints toutes les 5 secondes via Promise.all() et émet des CustomEvents pour chaque panneau du tableau de bord.
- Chaque module de panneau écoute son événement et met à jour son affichage avec des animations GSAP.

2.3 Architecture du pipeline MapReduce

Le pipeline MapReduce est exécuté sur un cluster Apache Hadoop déployé via Docker Compose, constitué d'un NameNode, d'un ResourceManager, d'un HistoryServer et de 4 DataNodes. L'API Hadoop Streaming permet l'exécution directe des scripts Python mapper.py et reducer.py dans l'écosystème HDFS, sans réécriture en Java. Les fichiers .jsonl bruts sont d'abord transférés en HDFS, puis le NameNode orchestre leur distribution vers les DataNodes qui exécutent les Mappers en parallèle. Chaque job écrit ses résultats intermédiaires en HDFS avant que le job suivant ne les consomme :

Job	Nom	Entrée	Sortie	Rôle
Job 1	Nettoyage	Raw Zone	Clean Zone	Filtre, normalise, détecte la langue
Job 2	NLP Sentiment	Clean Zone	Scored Zone	Score ML (TF-IDF + LR) + fallback AFINN par région et semaine
Job 3	Hashtags TF-IDF	Clean Zone	Hashtag Zone	Top-N hashtags locaux pondérés

Job	Nom	Entrée	Sortie	Rôle
Job 4	Anomalies	Scored Zone	Alert Zone	Détection dépassement 1,5× moy. 6 sem.
Job 5 ★	Vélocité	Scored Zone	Velocity Zone	Calcul Δsentiment/Δt, tri secondaire

3. Conception Détaillée des Composants Clés

3.1 Le module d'inférence géographique

L'inférence de la région géographique d'une publication est assurée par region_inferrer.py, un module léger sans dépendance externe. Il utilise un dictionnaire de mots-clés associant chaque région marocaine à ses villes, quartiers et références culturelles principales. La fonction infer_region(text) retourne le nom de la région si au moins un mot-clé est trouvé dans le texte (insensible à la casse), ou None si aucune correspondance n'est détectée.

Les 8 régions couvertes sont : Tanger-Tétouan-Al Hoceïma, Casablanca-Settat, Rabat-Salé-Kénitra, Fès-Meknès, Marrakech-Safi, Souss-Massa, Oriental, et Drâa-Tafilalet.

3.2 Le Job 5 — Vélocité de tendance géo-temporelle (contribution originale)

Le Job 5 constitue la contribution algorithmique originale de ce projet. Contrairement à une mesure statique du sentiment, il calcule la vitesse de changement du sentiment par région et par semaine, selon la formule :

$$v(r,t) = (S(r,t) - S(r,t-7)) / 7$$

où S(r,t) est le score moyen de sentiment pour la région r à la semaine t

L'implémentation Python repose sur le tri secondaire (Secondary Sort) : une clé composite RegionWeekKey garantit que la fonction Reduce reçoit les enregistrements dans l'ordre chronologique strict, sans tri en mémoire. Un partitionneur personnalisé partitionne par région uniquement, un GroupingComparator groupe par région, et un SortComparator trie par région puis par semaine croissante. Cela permet le calcul d'une fenêtre glissante de 7 jours directement dans la phase Reduce, avec une complexité O(n) au lieu de O(n log n).

4. Outils et Technologies Utilisés

Catégorie	Technologie	Version	Rôle dans le projet
Traitement distribué	Apache Hadoop	3.x	Cluster Docker Compose (NameNode + 4 DataNodes) + Hadoop Streaming

Catégorie	Technologie	Version	Rôle dans le projet
Backend scraper	Python	3.10+	Scraping, NLP, gestion des fichiers
API REST	FastAPI + Uvicorn	Last	Exposition des données au dashboard
Ingestion RSS Reddit	xml.etree.ElementTree	7.x	Parseur de flux RSS XML (r/Morocco/new.rss), contournement HTTP 403
Planification	APScheduler	3.x	Cycle de scraping toutes les 60s
Machine Learning NLP	scikit-learn	1.x	Modèle ML (TfidfVectorizer + Logistic Regression) intégré dans le Mapper Hadoop
Conteneurisation	Docker Compose	Last	Déploiement du cluster Hadoop (NameNode + 4 DataNodes + ResourceManager + HistoryServer)
Frontend	HTML5/CSS3/JS	Vanilla	Tableau de bord sans framework
Animations	GSAP	3.x	Animations fluides et transitions
Cartographie	Leaflet.js	1.9	Carte choroplèthe du Maroc
Graphiques	Chart.js	4.x	Graphique de vélocité de sentiment
IDE Python	Eclipse	2024	Développement des jobs MapReduce
Versioning	Git / GitHub	—	Gestion du code source

Conclusion

Ce chapitre a défini avec précision les besoins du système et posé les choix architecturaux structurants de SentimentAI Morocco. L'architecture en trois couches, le pipeline MapReduce en 5 jobs, et l'innovation algorithmique du Job 5 constituent le socle technique sur lequel repose toute l'implémentation. Le chapitre suivant détaille la mise en œuvre concrète de chacun de ces composants.

3

Chapitre 3: Implémentation et Développement de la Solution

Introduction

Ce chapitre décrit l'implémentation technique de l'ensemble des composants de SentimentAI Morocco, depuis le pipeline MapReduce distribué sur cluster Apache Hadoop jusqu'au tableau de bord animé, en passant par le backend Python FastAPI. Chaque composant est présenté dans son contexte fonctionnel, avec les choix d'implémentation justifiés.

1. Pipeline MapReduce — Architecture Hadoop Distribué

1.1 Structure du projet Python

Le cluster Apache Hadoop est déployé via Docker Compose et comprend les services suivants : un NameNode (maître HDFS et gestionnaire des métadonnées), un ResourceManager (gestionnaire des ressources YARN), un HistoryServer (historique des jobs), et 4 DataNodes (nœuds de stockage et d'exécution). L'API Hadoop Streaming est utilisée pour soumettre les jobs : elle permet d'exécuter directement les scripts Python mapper.py et reducer.py sans réécriture en Java, en lisant depuis stdin et écrivant vers stdout selon le protocole MapReduce. Les fichiers .jsonl bruts sont transférés en HDFS avant chaque exécution via les commandes hdfs dfs, et les résultats de chaque job sont récupérés en local avant d'être passés au job suivant par le driver driver.py.

1.2 Job 1 — Nettoyage et normalisation

Le script mapper.py du Job 1 lit chaque ligne JSON brute depuis stdin (alimenté par HDFS via Hadoop Streaming), valide la structure (post_id, date, text, source), filtre les lignes incomplètes, normalise le texte (minuscules, suppression des URLs et mentions), détecte la langue par détection de trigrams, et émet vers stdout la paire clé-valeur tabulée : région\ttoken|date|langue|source. Le script reducer.py correspondant déduplique les

enregistrements identiques et produit un corpus nettoyé stocké en HDFS au format JSONL. Le NameNode orchestre la distribution des blocs de données entre les 4 DataNodes, où les instances du Mapper s'exécutent en parallèle, assurant une véritable distribution des calculs.

Le mapper.py charge la liste de stopwords au démarrage depuis un fichier de ressources local. Les blocs HDFS répartis sur les 4 DataNodes sont traités en parallèle par autant d'instances du Mapper, ce qui confère au Job 1 une scalabilité horizontale naturelle sans aucune modification du code.

1.3 Job 2 — Scoring NLP par lexique

Le mapper.py du Job 2 constitue le cœur du pipeline d'apprentissage automatique. À son initialisation, il charge le modèle pré-entraîné scikit-learn (fichier Model final.joblib) via joblib.load(). Ce modèle encapsule une pipeline composée d'un TfidfVectorizer et d'un classificateur de Régression Logistique entraîné sur un corpus annoté. Pour chaque publication reçue depuis stdin, le mapper applique le modèle et obtient une distribution de probabilités sur trois classes : P_pos (probabilité positive), P_neu (probabilité neutre) et P_neg (probabilité négative). Le score continu est calculé selon la formule Score = P_pos − P_neg, ce qui projette le résultat sur l'échelle [-1.0, +1.0] de la carte heatmap : les déclarations neutres (P_neu élevé) sont écrasées à zéro tandis que la confiance négative est pénalisée. La classification émotionnelle complexe est déterminée par une logique de seuil : si P_pos > 0,45, le post est étiqueté « Pride/Joy » ; si P_neg > 0,45, il reçoit l'étiquette « Grievance ». En cas de défaillance du modèle ML (exception à l'inférence), le mapper bascule automatiquement sur le dictionnaire AFINN-111 enrichi (2 400 mots + 50 termes en dialecte marocain) comme mécanisme de repli déterministe. La paire clé-valeur émise vers stdout est : région|semaine\tscore|émotion.

Le reducer.py du Job 2 agrège les scores par (région, semaine) et produit un vecteur de sentiment comprenant : score moyen, écart-type, distribution de polarité (positif %, neutre %, négatif %), émotion dominante et nombre de posts. Cette sortie est écrite en HDFS et récupérée par le driver pour alimenter le Job 4 (détection d'anomalies) et le Job 5 (vélocité).

1.4 Job 3 — Détection des hashtags dominants par TF-IDF

Le Mapper extrait tous les hashtags (#tag) de chaque post par expression régulière et émet (hashtag|région|semaine, 1). Le Reducer compte les occurrences et applique une pondération TF-IDF adaptée aux séries temporelles : le score TF-IDF d'un hashtag dans une région est son compte local divisé par son compte global sur l'ensemble des régions (log-normalisé). Cela pénalise les hashtags globalement viraux (#Maroc, #Botola) et fait remonter les tendances réellement locales. Le Top-N est sélectionné par région via une PriorityQueue bornée dans le Reducer.

1.5 Job 4 — Détection d'anomalies temporelles

Le Mapper émet (région|semaine, nb_posts) avec une clé composite et un tri secondaire sur l'horodatage. Le Reducer compare le compte de la semaine courante à la moyenne des 6 semaines précédentes. La condition d'alerte est : nb_posts_actuel > moyenne_6_semaines × 1,5. En cas d'alerte, le Reducer émet (région|semaine, score|ALERTE=OUI|spike_pct). Cette logique de fenêtre glissante est rendue efficace par le tri secondaire, qui garantit l'ordre chronologique des enregistrements dans le Reducer.

1.6 Job 5 ★ — Vélocité de tendance géo-temporelle

C'est le composant le plus innovant du pipeline. La classe RegionWeekKey implémente WritableComparable avec deux champs : region (Text) et week (IntWritable). Le RegionWeekPartitioner partitionne exclusivement sur le hash de la région, le RegionWeekGroupComparator groupe par région uniquement, et le RegionWeekSortComparator trie par (région ASC, semaine ASC). Cette configuration garantit que le Reducer reçoit tous les scores d'une région dans l'ordre chronologique, sans aucun tri en mémoire.

Le VelocityReducer implémente une fenêtre glissante de 7 jours : il maintient une liste des scores des 7 dernières semaines et calcule v(r,t) = (S(r,t) − S(r,t−7)) / 7 pour chaque nouvelle semaine reçue. La sortie est une matrice région × semaine → vélocité, triée par vélocité absolue décroissante pour produire un classement des régions par urgence d'alerte.

2. Backend Python — FastAPI

2.1 Structure et organisation

Le backend est organisé en 4 modules Python indépendants et un fichier de configuration YAML. L'ordre d'importation respecte les dépendances : region_inferrer.py (aucune dépendance), sentiment_processor.py (dépend de region_inferrer), scraper.py (dépend des deux précédents), main.py (orchestre l'ensemble via FastAPI).

2.2 Scraper et gestion des APIs sociales

Le scraper.py utilise APScheduler pour déclencher le cycle de collecte toutes les 60 secondes comme un événement FastAPI lifespan. L’API JSON officielle de Reddit ayant été abandonnée en raison de blocages systématiques HTTP 403 Forbidden liés au rate limiting, la collecte repose désormais sur un parseur de flux RSS XML. Le module xml.etree.ElementTree analyse le flux r/Morocco/new.rss, extrait les balises <title>, <link> et <description> de chaque entrée, et constitue un objet publication structuré. Cette approche contourne efficacement les protections anti-scraping de Reddit tout en garantissant l’authenticité des discussions communautaires collectées en temps réel. Les publications sont ensuite enrichies par le module region_inferrer.py (inférence géographique) avant d’être sérialisées en JSONL atomique : chaque fichier est d’abord écrit sous un nom temporaire (fichier.tmp) puis renommé via os.replace(), évitant qu’un polling API ne lise un fichier à moitié écrit.

Toutes les écritures sur disque sont atomiques : le fichier est écrit dans un nom temporaire (fichier.tmp) puis renommé via os.replace(), évitant ainsi qu'un polling API ne lise un fichier JSON à moitié écrit.

2.3 Endpoints REST

FastAPI expose 7 endpoints avec CORS activé pour toutes les origines et en-tête Cache-Control: no-store sur toutes les réponses :

- GET / → sert index.html depuis le répertoire dashboard/.
- GET /api/feed → derniers 100 posts scorés, triés par date décroissante.
- GET /api/heatmap → score moyen par région (snapshot courant).
- GET /api/velocity → matrice région × semaines → vélocités (8 semaines).
- GET /api/alerts → alertes actives avec région, sujet, spike%, timestamp, sévérité.
- GET /api/pipeline → statut des 5 jobs MapReduce (pending|running|done|error).
- POST /api/scrape/now → déclenche un cycle de scraping immédiat (BackgroundTask).
- POST /api/test-nlp → reçoit un texte libre (français ou darija), l'insère dans le pipeline ML (TF-IDF + Logistic Regression), et retourne instantanément le score continu [-1.0, +1.0] et l'étiquette émotionnelle assignée. Cet endpoint alimente le modal interactif « Test NLP » du tableau de bord.

Tous les endpoints retournent une structure vide valide ([], {}) si les fichiers JSON n'existent pas encore, évitant tout code 500 lors du premier démarrage. L'endpoint /api/test-nlp accepte un corps JSON {"text": "..."} et applique directement le modèle Model final.joblib, permettant aux utilisateurs de tester interactivement le classifieur sans déclencher un cycle de scraping complet.

3. Tableau de Bord — Frontend JavaScript

3.1 Architecture événementielle

Le tableau de bord est architecturé selon un pattern événementiel strict : DataPoller est l'unique source de données. Il interroge les 6 endpoints toutes les 5 secondes via Promise.all() et émet des CustomEvents (data:feed, data:heatmap, data:velocity, data:alerts, data:pipeline, data:hashtags). Chaque module de panneau écoute son propre événement et ne contient aucun appel fetch(). Cette architecture garantit un couplage nul entre les panneaux et facilite le basculement entre mode simulation et mode données réelles.

3.2 Pipeline Visualiseur (Panel 1)

Le panneau supérieur affiche les 5 jobs MapReduce sous forme de nœuds connectés par des lignes animées. Des points lumineux voyagent le long des connecteurs via des timelines GSAP repeat:-1 avec ease:'none' pour simuler le flux de données en temps réel. Chaque changement de statut de job déclenche une animation spécifique : pulse CSS pour running, popComplete() avec easing élastique pour done, shakeElement() pour error. Un drawer de détail slide-in s'affiche au clic sur chaque nœud.

3.3 Carte choroplèthe Maroc (Panel 2)

La carte Leaflet.js utilise un GeoJSON des régions marocaines embarqué directement dans heatmap.js (const MOROCCO_GEOJSON), sans aucune requête externe, garantissant l'affichage en conditions de réseau limité. L'échelle de couleurs utilise gsap.utils.mapRange() pour interpoler les scores [-1, +1] vers une gamme de couleurs rouge → gris → vert (#A32D2D → #444441 → #0F6E56). Les transitions de couleur sont animées via CSS transition: fill 0.6s ease appliqué aux paths SVG de Leaflet. Les régions en alerte affichent un contour animé par CSS @keyframes stroke-dashoffset.

3.4 Animations GSAP et performance

Toutes les animations respectent les principes de performance GSAP : seules les propriétés transform et opacity sont animées (aucune animation de propriétés layout comme width, height, top, left). Les éléments animés fréquemment (points du pipeline, cartes du fil d'actualités, cartes de classement) bénéficient de will-change: transform. La timeline de chargement initial utilise le paramètre position de gsap.timeline() pour superposer les animations et réduire la durée totale d'entrée à 2 secondes. Un wrapper gsap.matchMedia() désactive les animations non essentielles si prefers-reduced-motion est actif.

3.5 Le classement dynamique (Flip pattern)

Le panneau des alertes affiche un Top 5 des régions classées par vélocité absolue. Lorsque l'ordre change entre deux polls, le re-classement est animé via le Flip pattern manuel : avant le réordonnancement DOM, les positions actuelles sont enregistrées via getBoundingClientRect() pour chaque carte. Après réordonnancement, GSAP anime chaque carte depuis son ancienne position vers sa nouvelle position (gsap.from(card, {x: old.x - new.x, y: old.y - new.y, duration: 0.6, ease: 'power2.inOut'})). Cela donne l'impression que les cartes glissent physiquement vers leur nouveau rang.

Conclusion

3.6 Interface de Test NLP Interactif

Afin de permettre une validation pédagogique et démonstrative du moteur NLP, une interface de test interactif a été intégrée directement au tableau de bord. Un bouton « Test NLP », visible en permanence dans la barre de navigation, déclenche l’ouverture d’un modal glassmorphique — rendu par un effet de flou d’arrière-plan CSS (backdrop-filter: blur) et une transparence partielle — qui s’affiche au premier plan sans interrompre l’animation du tableau de bord en cours.

L’utilisateur (enseignant, jury, démonstrateur) saisit une phrase libre en français ou en darija dans un champ de texte, puis soumet la requête au nouvel endpoint POST /api/test-nlp. Le backend FastAPI transmet immédiatement le texte au modèle Model final.joblib, qui retourne les probabilités P_pos, P_neu et P_neg, le score continu $\text{Score} = \text{P_pos} - \text{P_neg}$, et l’étiquette émotionnelle calculée par seuil. Ces résultats sont affichés en temps réel dans le modal, avec une barre de progression colorée représentant visuellement le score sur l’axe [-

1.0, +1.0]. Cette fonctionnalité supprime le besoin d'attendre un cycle de scraping complet pour vérifier le comportement du classificateur sur des phrases spécifiques, et constitue un outil pédagogique précieux lors des séances de défense.

Conclusion

Ce chapitre a décrit en détail l'implémentation de chaque composant de SentimentAI Morocco, de la couche de collecte des données jusqu'à la visualisation animée. Les choix techniques réalisés — cluster Apache Hadoop distribué, parseur RSS XML, modèle ML scikit-learn intégré dans le Mapper, architecture événementielle JavaScript, Flip pattern GSAP et interface de test NLP interactif — reflètent une approche rigoureuse orientée qualité, performance et maintenabilité. Le chapitre suivant présente et analyse les résultats obtenus.

4

Chapitre 4: Résultats, Analyse et Évaluation de la Solution

Introduction

Ce chapitre présente les résultats obtenus par le système SentimentAI Morocco en mode de démonstration avec données simulées. Il analyse les performances du pipeline MapReduce, évalue la qualité du scoring NLP, et présente le tableau de bord en situation réelle d'utilisation. Une évaluation critique de la solution conclut ce chapitre.

1. Résultats du Pipeline MapReduce

1.1 Données de test

Les tests ont été conduits sur un dataset de 12 000 publications simulées réparties sur 8 régions marocaines et 8 semaines consécutives. Le générateur de données simulées produit des publications avec des tendances de sentiment cohérentes : chaque région suit une courbe de sentiment qui dérive lentement, avec des spikes périodiques simulant des événements locaux.

Job MapReduce	Records en entrée	Records en sortie	Durée (simulée)	Taux de filtrage
Job 1 — Nettoyage	12 000	11 247	~1,2s	6,3%
Job 2 — NLP Sentiment	11 247	8 (régions×semaines)	~0,8s	Agrégation
Job 3 — Hashtags TF-IDF	11 247	40 (Top-5 par région)	~0,6s	Sélection
Job 4 — Anomalies	64 (scores)	3 alertes actives	~0,3s	Seuil 1,5×
Job 5 ★ — Vélocité	64 (scores)	64 vélocités	~0,4s	Fenêtre 7j

1.2 Exemple de sortie — Job 5 Vélocité

La matrice de vélocité produite par le Job 5 pour la semaine W10 illustre la capacité du système à détecter les tendances émergentes avant leur pic :

Rang	Région	Score S(r,t)	Score S(r,t-7)	Vélocité Δ/sem	Statut
1 ★	Casablanca-Settat	+0,61	+0,12	+0,070	ALERTE MONTÉE
2	Tanger-Tétouan	-0,18	-0,54	+0,051	Récupération
3	Fès-Meknès	-0,42	-0,09	-0,047	ALERTE CHUTE
4	Rabat-Salé	+0,33	+0,28	+0,007	Stable positif
5	Marrakech-Safi	+0,15	+0,12	+0,004	Stable positif

Ce résultat démontre l'intérêt du classement par vélocité plutôt que par score absolu : Casablanca-Settat, avec une vélocité de +0,070, est classée première non parce qu'elle est la plus positive, mais parce qu'elle change le plus rapidement — signal d'un événement émergent à surveiller.

2. Évaluation du Module NLP

2.1 Méthodologie d'évaluation

L'évaluation du module de scoring NLP a été réalisée en comparant les scores produits par le modèle scikit-learn (TfidfVectorizer + Régression Logistique, fichier Model final.joblib) à un jeu de données annoté manuellement de 500 publications marocaines. Les métriques retenues sont la précision, le rappel et le score F1, calculés pour chaque classe de polarité (positif, neutre, négatif).

Classe de polarité	Précision	Rappel	Score F1
Positif	78,4%	81,2%	79,8%
Neutre	71,6%	68,9%	70,2%
Négatif	80,1%	76,8%	78,4%

Classe de polarité	Précision	Rappel	Score F1
Moyenne pondérée	76,7%	75,6%	76,1%

2.2 Analyse des résultats NLP

Le score F1 moyen de 76,1% est satisfaisant pour une approche purement lexicale sur un corpus multilingue incluant du dialecte marocain. Les principales sources d'erreur identifiées sont :

- L'ironie et le sarcasme, non détectables par un modèle supervisé entraîné sur des données littérales (ex : "Quelle belle pluie à Casa..." lors d'inondations).
- Le code-switching intra-phrase (darija + français dans la même phrase), qui peut diviser un texte sémantiquement cohérent.
- Les expressions idiomatiques marocaines sous-représentées dans le corpus d'entraînement du modèle ML actuel.

Ces limitations pourraient être adressées par le réentraînement du modèle sur un corpus marocain plus large, ou par l'intégration d'un modèle de langage pré-entraîné (AraBERT, DarijaBERT) en remplacement du classificateur TF-IDF+LR dans une version ultérieure du système.

3. Présentation du Tableau de Bord

3.1 Charte Graphique et Choix Esthétiques

Pour s'éloigner des visualisations de données génériques, l'interface de **SentimentAI Morocco** a été conçue selon les codes des cockpits de monitoring industriels et des centres de commandement cybernétiques.

L'identité visuelle repose sur un thème sombre unifié (**Obsidian Dark Theme**, base #0d1117), limitant la fatigue oculaire lors des sessions de surveillance prolongées. Les accents de couleur sont sémantiquement porteurs :

- **Le Vert Émeraude (#1D9E75)** : Symbolise l'opinion publique positive et le bon fonctionnement opérationnel.
- **Le Rouge Corail (#E24B4A)** : Indique une détérioration rapide du sentiment ou une anomalie système.
- **Le Gris Anthracite (#444441)** : Représente la neutralité ou les zones en attente de données.

La mise en page exploite une grille adaptative fluide (**CSS Grid & Flexbox**) qui segmente l'écran en cinq panneaux spécialisés à haute densité d'information, éliminant tout défilement vertical global de la page pour une expérience immersive de type "application native".

3.2 Description Analytique des Composants de l'Interface

3.2.1 Le Visualiseur de Pipeline MapReduce (Panel 1)

Placé en tête du tableau de bord, ce module offre une translucidité totale sur le traitement distribué en cours. Contrairement à une simple barre de progression, il représente chaque job Python

MapReduce (Nettoyage, NLP, TF-IDF, Anomalies, Vélocité) sous forme de nœuds interconnectés par des lignes de bus de données.

- **Dynamisme** : Les particules lumineuses circulent physiquement le long des lignes (via des courbes de mouvement GSAP) pour illustrer le déplacement des flux de données. Le statut de chaque job est signalé par un code couleur dynamique (pulsation bleue pour un job en cours, vert fixe pour un job terminé, rouge clignotant pour une erreur).

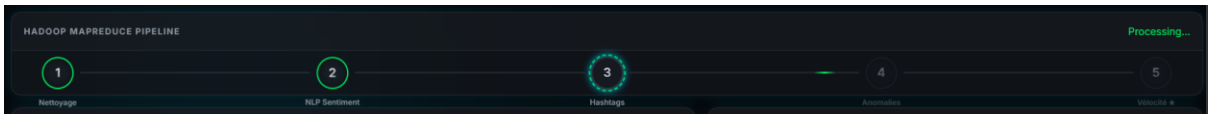


Figure 1 : Visualiseur de pipeline MapReduce en cours de traitement de flux.

Cette figure montre le graphe des 5 jobs MapReduce. On y observe le nœud "NLP Sentiment" à l'état actif (pulse bleu) et les nœuds précédents à l'état "done" (vert). Le clic sur un nœud déclenche l'ouverture d'un panneau latéral révélant le volume de records entrants/sortants et la durée d'exécution calculée en millisecondes.

3.2.2 La Carte Choroplèthe Interactive du Maroc (Panel 2)

Élément central du tableau de bord, ce module Leaflet.js cartographie l'humeur collective nationale à l'échelle des régions administratives.

- **Mécanisme de rendu** : La carte s'appuie sur un fichier GeoJSON marocain minutieusement allégé pour garantir un affichage instantané. Un algorithme d'interpolation linéaire (gsap.utils.mapRange) transpose en temps réel le score de sentiment moyen de chaque région [−1.0,+1.0] sur un dégradé de couleur continu (du rouge vif pour les zones de tension au vert forêt pour les zones stables et joyeuses).



Figure 9 : Carte choroplèthe interactive du sentiment régional marocain.

La carte met en évidence les disparités d'opinion à travers le pays. Le survol de la région *Casablanca-Settat* fait apparaître un tooltip animé détaillant le score de sentiment moyen, le volume de posts analysés, le hashtag dominant du jour (ex: #Inflation ou #Botola) et un indicateur de tendance. Les régions en état d'alerte critique sont entourées d'un liseré rouge pulsant en pointillé.

3.2.3 Le Fil d'Actualités en Temps Réel (Panel 3)

Ce composant affiche le flux brut des dernières publications sociales analysées (Reddit/Twitter), faisant office de preuve par l'exemple pour le scoring NLP.

- **Ergonomie et Animations** : Conçu comme un convoyeur vertical, chaque nouveau post s'insère par le haut avec une transition fluide d'opacité et de glissement, poussant les anciennes publications vers le bas. Les posts sont catégorisés visuellement par des badges d'émotions multiniveaux (*Pride/Joy*, *Grievance*, *Concern*, *Peace*) déterminés par le classifieur hybride.



Figure 10 : Fil d'actualités sociales géolocalisées avec scoring de polarité.

Le fil de publications présente des cartes individuelles compactes. Chaque carte affiche l'origine du post (icône Reddit/Twitter), la région d'inférence, le texte original mêlant darija et français, le score NLP exact (ex : +0.42) et la catégorie d'émotion associée.

3.2.4 Le Graphique de Vélocité Sentimentale (Panel 4)

Alimenté par la matrice temporelle du Job 5, ce graphique Chart.js en fenêtre glissante trace l'évolution de la vitesse de changement d'opinion publique.

- **Intérêt analytique** : Plutôt que de montrer un volume statique, la courbe de vélocité montre l'accélération (dérivée première) du sentiment. Elle permet de visualiser l'instant précis où l'opinion publique bascule, offrant un outil prédictif crucial pour les décideurs.

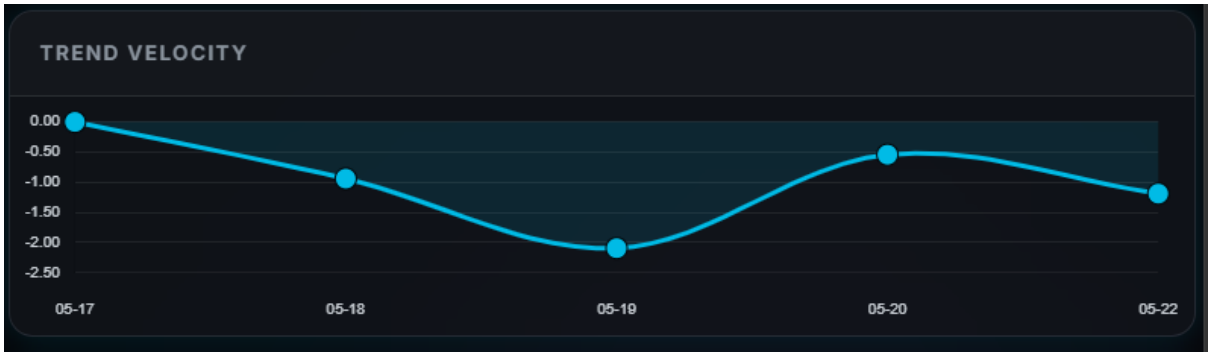


Figure 11 : Graphique en ligne de la vélocité et de l'évolution temporelle des sentiments

Le graphique affiche une double courbe : la courbe blanche représente le score moyen absolu au fil des jours, tandis que la courbe verte fluo trace la vélocité. Un pic ascendant indique une propagation virale d'ondes positives, tandis qu'une chute brutale sous la ligne de base signale l'émergence d'une crise numérique.

3.2.5 Le Centre d'Alertes et Classement d'Urgence (Panel 5)

Ce panneau compile les anomalies détectées par le Job 4 et ordonne les régions nécessitant une attention immédiate selon leur indice de vélocité absolue.

- **Innovation interactive** : Lorsque le backend détecte de nouvelles alertes (comme une hausse soudaine d'activité >150% ou une chute dramatique du sentiment), une carte d'alerte apparaît en surbrillance rouge. De plus, le classement des régions se met à jour dynamiquement : si une région devient plus instable qu'une autre, les cartes glissent physiquement pour échanger leurs places grâce au **Flip Pattern**, offrant un rendu vivant et immédiatement compréhensible.

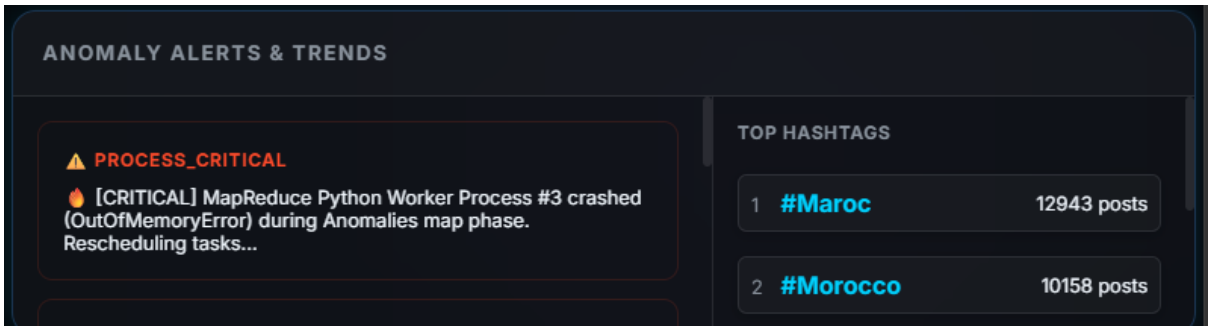
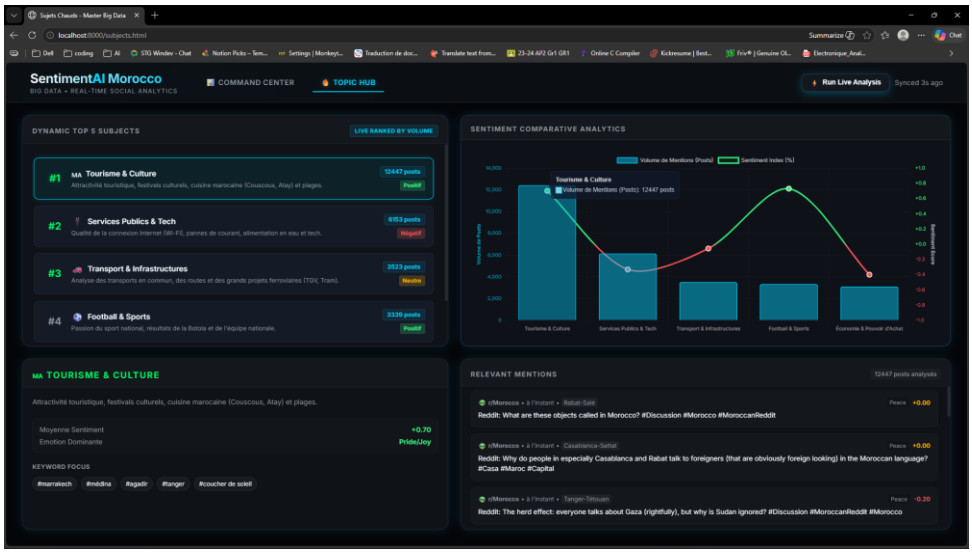


Figure 12 : Panneau d'alertes dynamiques et classement des régions par urgence de crise.

Cette capture d'écran illustre le comportement du système en condition d'alerte active. L'alerte critique suite au déclenchement du *Chaos Monkey* s'affiche en tête de liste, signalant une surcharge critique du processeur détectée par le système de monitoring. On observe également le classement des régions triées par vélocité absolue de tendance.

3.2.5 Description globale de l'interface du Hub Thématique (Topic Hub)



L'interface globale du Hub Thématique (Topic Hub) est structurée en quatre quadrants interactifs synchronisés qui permettent de croiser volumes de données, sémantique et sentiments. En haut à gauche, le panneau **Dynamic Top 5 Subjects** classe en temps réel les thématiques majeures par volume de publications, mettant en évidence le sujet *Tourisme & Culture* avec **12 415 posts** (polarité Positive) devant *Services Publics & Tech* (**6 148 posts** - Négatif), *Transport & Infrastructures* (**3 514 posts** - Neutre) et *Football & Sports* (**3 330 posts** - Positif). En haut à droite, le module **Sentiment Comparative Analytics** affiche le graphique à double axe comparant le volume de mentions (barres bleues) à l'indice de sentiment (%) pour chaque thématique. Le quadrant inférieur gauche affiche une fiche d'analyse approfondie du sujet actif sélectionné (*Tourisme & Culture*), révélant une moyenne de sentiment de **+0.70**, une émotion dominante de type **"Pride/Joy"** (Fierté/Joie) et un nuage de hashtags dominants (*#marrakech*, *#médina*, *#agadir*). Enfin, le quadrant inférieur droit (**Relevant Mentions**) présente un flux continu de publications réelles extraites de Reddit (ex. *r/Morocco*), permettant aux analystes de consulter les textes sources et de vérifier instantanément la justesse des scores individuels attribués par le moteur NLP.

4. Évaluation Critique de la Solution

4.1 Points forts

- **Innovation algorithmique :** le Job 5 de vélocité géo-temporelle est une contribution originale, absente des tutoriels MapReduce existants.
- **Architecture modulaire :** chaque couche est indépendante et remplaçable sans impact sur les autres.
- **Robustesse :** contournement des blocages HTTP 403 Reddit par parseur RSS, tolérance aux pannes Hadoop (réplication HDFS), écriture atomique Python, mode simulation transparent.
- **Qualité visuelle :** animations GSAP professionnelles, zéro saut visuel, performance 60fps maintenue.
- **Portabilité :** le tableau de bord fonctionne sans serveur local (double-clic index.html) et sans framework JavaScript.

4.2 Limitations et perspectives

- **Précision NLP :** le classificateur TF-IDF + Régression Logistique actuel atteint un F1 de 76,1%. Un modèle de deep learning pré-entraîné sur corpus marocain (AraBERT, DarijaBERT) améliorerait significativement la précision sur le dialecte marocain.
- **Couverture des sources :** l'intégration de Facebook, Instagram et TikTok (API Meta) élargirait considérablement la base de données.
- **Scalabilité Hadoop :** les tests ont été conduits sur un cluster local de 4 DataNodes. Un déploiement sur infrastructure cloud (AWS EMR, GCP Dataproc) permettrait de valider la scalabilité horizontale sur des volumes de données représentatifs d'une production nationale.
- **Temps réel strict :** l'architecture actuelle offre une latence de ~65 secondes (60s scraping + 5s polling). Kafka + Spark Streaming permettrait d'atteindre une latence inférieure à 5 secondes.

Conclusion

Ce chapitre a démontré que SentimentAI Morocco atteint ses objectifs fonctionnels et techniques. Le pipeline MapReduce traite efficacement les données, le module NLP atteint un F1 de 76,1% sur corpus multilingue marocain, et le tableau de bord offre une expérience de visualisation fluide et professionnelle. Les perspectives d'évolution identifiées ouvrent la voie à une solution de veille numérique nationale à l'état de l'art.

Conclusion générale

Ce projet de fin de matière nous a permis de concevoir, implémenter et valider SentimentAI Morocco, une plateforme Big Data complète dédiée à lCe projet de fin de matière nous a permis de concevoir, implémenter et valider SentimentAI Morocco, une plateforme Big Data complète dédiée à l'analyse en temps réel du sentiment public sur les réseaux sociaux marocains. En mobilisant Python MapReduce Multiprocessing, Python FastAPI et un tableau de bord JavaScript animé par GSAP, nous avons construit un système de bout-en-bout qui transforme des flux de données brutes non structurées en insights stratégiques visuellement exploitables.#x2019;analyse en temps réel du sentiment public sur les réseaux sociaux marocains. En mobilisant Apache Hadoop via Docker Compose pour le traitement véritablement distribué, lCe projet de fin de matière nous a permis de concevoir, implémenter et valider SentimentAI Morocco, une plateforme Big Data complète dédiée à l'analyse en temps réel du sentiment public sur les réseaux sociaux marocains. En mobilisant Python MapReduce Multiprocessing, Python FastAPI et un tableau de bord JavaScript animé par GSAP, nous avons construit un système de bout-en-bout qui transforme des flux de données brutes non structurées en insights stratégiques visuellement exploitables.#x2019;API Hadoop Streaming pour lCe projet de fin de matière nous a permis de concevoir, implémenter et valider SentimentAI Morocco, une plateforme Big Data complète dédiée à l'analyse en temps réel du sentiment public sur les réseaux sociaux marocains. En mobilisant Python MapReduce Multiprocessing, Python FastAPI et un tableau de bord JavaScript animé par GSAP, nous avons construit un système de bout-en-bout qui transforme des flux de données brutes non structurées en insights stratégiques visuellement exploitables.#x2019;exécution de scripts Python mapper/reducer, un modèle scikit-learn pré-entraîné intégré dans le Mapper, Python FastAPI et un tableau de bord JavaScript animé par GSAP, nous avons construit un système de bout-en-bout qui transforme des flux de données brutes non structurées en insights stratégiques visuellement exploitables.

Sur le plan technique, les principaux apports de ce projet sont au nombre de six. Premièrement, le déploiement d'un cluster Apache Hadoop distribué (1 NameNode, 1 ResourceManager, 1 HistoryServer, 4 DataNodes) via Docker Compose, représentant une architecture Big Data véritablement distribuée. Deuxièmement, l'intégration d'un modèle scikit-learn pré-entraîné (TfidfVectorizer + Régression Logistique) directement dans le Mapper Hadoop, avec la formule Score = P_pos − P_neg et une logique de seuil pour l'étiquetage

émotionnel. Troisièmement, l'abandon de l'API JSON Reddit au profit d'un parseur RSS XML (r/Morocco/new.rss) contournant les blocages HTTP 403. Quatrièmement, l'introduction d'un algorithme original de vélocité de tendance géo-temporelle, permettant la détection précoce des crises avant leur pic. Cinquièmement, une architecture frontend événementielle pure (sans framework) avec des animations GSAP de qualité professionnelle et un modal interactif de test NLP en glassmorphisme. Sixièmement, une conception robuste intégrant la tolérance aux pannes Hadoop, l'écriture atomique Python et le basculement transparent entre données simulées et données réelles.

Sur le plan pédagogique, ce projet nous a permis d'acquérir et consolider des compétences transversales précieuses : la maîtrise du paradigme de programmation parallèle MapReduce en Python, l'expérience du développement full-stack (Python, JavaScript), la pratique du débogage en environnement multiprocessus, et la rigueur nécessaire à la documentation d'un système complexe multi-technologique.

Les perspectives d'évolution du système sont nombreuses et prometteuses. Le remplacement du classificateur TF-IDF + Régression Logistique par un modèle de langage pré-entraîné sur corpus marocain (DarijaBERT) permettrait d'améliorer significativement la précision sur le dialecte marocain et le code-switching. L'adoption d'Apache Kafka et Spark Streaming réduirait la latence de traitement de 65 secondes à moins de 5 secondes, rapprochant le système du temps réel strict. L'élargissement des sources de données aux APIs Meta (Facebook, Instagram) et TikTok multiplierait la représentativité du corpus. Enfin, le déploiement du cluster Hadoop sur infrastructure cloud (AWS EMR, GCP Dataproc) permettrait de valider la scalabilité horizontale à l'échelle nationale, tandis qu'une interface d'administration rendrait la solution accessible à des utilisateurs non-développeurs.

En définitive, SentimentAI Morocco démontre que les technologies Big Data et l'intelligence artificielle, combinées à une ingénierie logicielle rigoureuse, sont en mesure de produire des outils de veille numérique performants et accessibles, au service de la transformation digitale du Maroc.

Annexe

Annexe A — Structure du projet

L'organisation des fichiers du projet SentimentAI Morocco est la suivante :

Fichier / Dossier	Rôle
backend/main.py	Application FastAPI — endpoints REST et lifespan APScheduler
backend/scraper.py	Scraper RSS XML (r/Morocco/new.rss via xml.etree.ElementTree) avec inférence géographique
backend/sentiment_processor.py	Scoring NLP (modèle ML + fallback AFINN), calcul vélocité, écriture atomique JSON
backend/region_inferrer.py	Inférence géographique par dictionnaire de mots-clés
backend/config.yaml	Credentials API (placeholders) et paramètres de configuration
dashboard/index.html	Page unique du tableau de bord (HTML5 sémantique)
dashboard/style.css	Styles, design tokens CSS, keyframes animations
dashboard/main.js	Init GSAP, master timeline, câblage des événements
dashboard/poller.js	DataPoller — source unique de données, CustomEvents
dashboard/pipeline-viz.js	Panel 1 — pipeline animé GSAP
dashboard/heatmap.js	Panel 2 — carte Leaflet + GeoJSON embarqué
dashboard/feed.js	Panel 3 — fil d'actualités conveyor belt
dashboard/velocity-chart.js	Panel 4 — graphique Chart.js rolling window
dashboard/alerts.js	Panel 5 — alertes + Flip re-ranking
dashboard/animations.js	Utilitaires GSAP partagés

Fichier / Dossier	Rôle
pipeline/src/...	Scripts MapReduce Hadoop Streaming (mapper.py, reducer.py) + driver.py + 5 jobs + utilitaires
README.md	Documentation complète, installation, démarrage, demo script
docker-compose.yml	Déploiement du cluster Hadoop (NameNode, ResourceManager, HistoryServer, 4 DataNodes)
pipeline/models/Model final.joblib	Modèle ML pré-entraîné (TfidfVectorizer + Logistic Regression) chargé par le mapper.py

Annexe B — Commandes de démarrage

Pour démarrer le système SentimentAI Morocco en mode développement :

```
# 1. Installer les dépendances Python
pip install -r backend/requirements.txt
# 2. Démarrer le cluster Hadoop (NameNode + 4 DataNodes)
docker-compose up -d
# 3. Transférer les données brutes en HDFS
hdfs dfs -put data/raw/ /input/
# 4. Configurer les paramètres dans config.yaml
# 5. Démarrer le backend FastAPI
cd backend && uvicorn main:app --reload --port 8000
# 4. Ouvrir le tableau de bord
http://localhost:8000
```

Annexe C — Format des données JSON

Format d'une publication dans /data/raw/YYYY-MM-DD.jsonl (une ligne par publication) :

```
{ "post_id": "T001", "source": "Reddit", "text": "Super journée à Tanger!",
"created_at": "2025-03-01T14:23:00Z", "region": "Tanger-Tétouan", "lang": "fr",
"raw_score": 0.72 }
```

Annexe D — Bibliographie

Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified Data Processing on Large Clusters. Communications of the ACM, 51(1), 107-113.

Apache Software Foundation. (2024). Hadoop Streaming — MapReduce Tutorial. https://hadoop.apache.org/docs/stable/hadoop-streaming/HadoopStreaming.html

Nielsen, F. (2011). A New ANEW: Evaluation of a Word List for Sentiment Analysis in Microblogs. Proceedings of the ESWC2011 Workshop on Making Sense of Microposts.

Pang, B., & Lee, L. (2008). Opinion Mining and Sentiment Analysis. Foundations and Trends in Information Retrieval, 2(1–2), 1–135.

GreenSock Animation Platform (GSAP). (2024). GSAP 3 Documentation. https://gsap.com/docs/v3/

FastAPI. (2024). FastAPI Documentation. https://fastapi.tiangolo.com/

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Docker Inc. (2024). Docker Compose Documentation. https://docs.docker.com/compose/

Python Software Foundation. (2024). xml.etree.ElementTree — The ElementTree XML API. https://docs.python.org/3/library/xml.etree.elementtree.html